AWS VPC PEERING PROJECT

Private Connectivity Between Two Amazon VPCs

SUBMISSION DETAIL	VALUE
Submitted by	Astha Verma
Batch	2024–2028
Project	EWB AWS VPC Peering Student Project
Cloud platform	Amazon Web Services
Requested documentation region	Asia Pacific (Mumbai) — ap-south-1
Report date	31 October 2026

Project overview

This report describes the design and workflow for private communication between two Amazon Virtual Private Clouds using AWS VPC Peering. It covers network architecture, subnet addressing, EC2 deployment, route tables, security groups, bastion access, connectivity checks and cleanup.

Region note: Mumbai (ap-south-1) is the requested region for this document. The AWS console screenshots and resource identifiers supplied for this report were captured in Sydney (ap-southeast-2). They are retained as historical evidence and must not be interpreted as proof that those same resources exist in Mumbai.

1. ABSTRACT AND OBJECTIVES

The project demonstrates private connectivity between two VPCs through a peering connection. VPC A contains a public subnet for a bastion host and a private subnet for App-A. VPC B contains a private subnet for App-B. Routes on both sides provide forward and return paths, while security groups restrict SSH and ICMP traffic.

• Create two VPCs with non-overlapping CIDR blocks.

• Configure public and private subnets.

• Establish and accept a VPC Peering connection.

• Configure routes in both directions.

• Restrict network access using security groups.

• Validate ICMP communication and SSH through a bastion host.

• Perform controlled failure/recovery testing and clean up resources.

2. NETWORK ARCHITECTURE

Component	CIDR / IP	Purpose
VPC A	10.10.0.0/16	Primary VPC
VPC A public subnet	10.10.1.0/24	Bastion subnet
VPC A private subnet	10.10.2.0/24	App-A subnet
VPC B	10.20.0.0/16	Peer VPC
VPC B private subnet	10.20.1.0/24	App-B subnet
Bastion (observed)	10.10.1.96 private; 3.27.190.136 public	Existing EC2 evidence
App-A / App-B	10.10.2.10 / 10.20.1.10	Workbook design targets

```
Internet → Internet Gateway → VPC A public subnet
                            → Bastion host
VPC A private subnet / App-A ← VPC Peering → VPC B private subnet / App-B
```

3. AWS RESOURCE CONFIGURATION

Resource	Observed evidence / design
VPC A	ewb-vpc-a · vpc-0f0beaf14cb3bfe63
VPC B	ewb-vpc-b · vpc-02444678518f0678e
VPC A public subnet	subnet-007eb456013dd2948 · 10.10.1.0/24
VPC A private subnet	subnet-027fc05434ffb3005 · 10.10.2.0/24
VPC B private-design subnet	subnet-01eaae7e86746f412 · 10.20.1.0/24
EC2 instance	i-01f09ec5463007f12 · t3.micro · Amazon Linux 2023
Peering reference	pcx-02a9b319c878a2088

The observed instance was Running with 3/3 status checks. EC2 Instance Connect was established. Apache was installed and observed active, and a custom page was accessible at http://3.27.190.136. These identifiers and screenshots originate from the Sydney console session.

4. ROUTE TABLE CONFIGURATION

Route table	Required routes
ewb-a-public-rt	10.10.0.0/16 → local; 0.0.0.0/0 → Internet Gateway; 10.20.0.0/16 → peering
ewb-a-private-rt	10.10.0.0/16 → local; 10.20.0.0/16 → peering
ewb-b-private-rt	10.20.0.0/16 → local; 10.10.0.0/16 → peering

The VPC B private route table was observed with 10.20.0.0/16 → local and 10.10.0.0/16 → peering, both Active. Its latest explicit subnet association was subnet-01eaae7e86746f412.

5. SECURITY GROUP POLICY

Target	Protocol / port	Source
Bastion	SSH TCP 22	Administrator public IPv4 /32
App-A	SSH TCP 22	Bastion private IP /32
App-A	ICMP Echo Request	App-B private IP /32
App-B	SSH TCP 22	Bastion private IP /32
App-B	ICMP Echo Request	App-A private IP /32

6. BASTION AND PRIVATE ACCESS

The bastion host is the controlled SSH entry point. Private instances are intended to remain without public IPv4 addresses. SSH ProxyJump can be configured using the bastion as the jump host.

```
Host ewb-bastion
  HostName 3.27.190.136
  User ec2-user
  IdentityFile ~/.ssh/ewb-key.pem

Host ewb-app-a
  HostName 10.10.2.10
  User ec2-user
  ProxyJump ewb-bastion

Host ewb-app-b
  HostName 10.20.1.10
  User ec2-user
  ProxyJump ewb-bastion
```

7. CONNECTIVITY VALIDATION

Test	Command	Expected result
Forward ping	ping -c 4 10.20.1.10	Replies after routes and ICMP rules are configured
Reverse ping	ping -c 4 10.10.2.10	Replies after return route and ICMP rule
SSH to App-B	ssh ewb-app-b	Private SSH through bastion
Controlled failure	Remove App-B ICMP rule; repeat forward ping	Ping fails while rule is absent
Recovery	Restore ICMP rule; repeat ping	Ping succeeds again

These are validation procedures and expected outcomes. The supplied screenshots do not establish successful cross-VPC ping, ProxyJump, or controlled ICMP failure/recovery. Capture actual terminal evidence before marking these tests complete.

8. TROUBLESHOOTING

• Confirm peering connection state is Active.

• Verify remote VPC routes in both route tables and subnet associations.

• Check security-group inbound source CIDRs and host firewall rules.

• Use private IPv4 addresses for inter-VPC tests.

• Check SSH key path, bastion access and TCP 22 permissions.

9. CLEANUP

• Terminate lab EC2 instances after evidence capture.

• Remove unused EBS volumes, snapshots and Elastic IP addresses.

• Delete peering connection and peering routes.

• Delete lab security groups, subnets and custom route tables.

• Detach and delete the Internet Gateway, then delete the lab VPCs.

• Review Billing for remaining resources.

10. LEARNING OUTCOMES AND CONCLUSION

The project reinforces VPC CIDR planning, subnetting, route-table configuration, VPC Peering, stateful security groups, bastion-based administration and network troubleshooting. Successful peering requires an Active connection, correct routes in both directions, correct subnet associations and security rules that permit the intended traffic.

Author: Astha Verma
Project: AWS VPC Peering Lab
Requested documentation region: Mumbai, ap-south-1